Problemes 1: Programació en C++

Estructura de Dades - Curs 2025/2026
Grau d'Enginyeria Informàtica
Facultat de Matemàtiques i Informàtica

Objectiu

Aquests problemes són opcionals, però us poden ajudar molt a guanyar confiança i destresa amb el C++. Estan dissenyats perquè pugueu practicar de manera progressiva i consolidar els conceptes bàsics del llenguatge. Us animem a fer-ne alguns —o tots!— seguint l'ordre suggerit, ja que us facilitarà molt el camí mentre aneu aprenent.

1 Problemes bàsics de llenguatge C++

1. Escriu un programa `Fun.cpp`. Creeu un mètode `main()` que retorni l'enter 0 i escrigui per pantalla:

```
 _   _ ______  ______  _____  _____  _   __ _____  _
| | | || ___ \ | ___ \|  _  |/  __ \| | / //  ___|| |
| | | || |_/ / | |_/ /| | | || /  \/| |/ / \ `--. | |
| | | || ___ \ |    / | | | || |    |    \  `--. \| |
| |_| || |_/ / | |\ \ \ \_/ /| \__/\| |\  \/\__/ /|_|
 \___/ \____/  \_| \_| \___/  \____/\_| \_/\____/ (_)
```

```cpp
#include <iostream>

int main() {
    // Write your code here
    return 0;
}
```

2. Escriu un programa amb un `main()` que demani a l'usuari el seu pes en kg i cridi un mètode `toPounds(float kg)` que retorni el pes en pounds.

3. Escriu un programa amb un `main()` que demani a l'usuari un número i indiqui per pantalla si el nombre és parell o senar.

4. Escriu un programa que calculi el sou que li correspon a un treballador d'una empresa que cobra 40.000 euros anuals, el programa ha de realitzar els càlculs en funció dels següents criteris:

   - Si porta més de 10 anys en l'empresa se li aplica un augment del 10%.
   - Si porta menys de 10 anys però més que 5 se li aplica un augment del 7%.
   - Si porta menys de 5 anys però més que 3 se li aplica un augment del 5%.
   - Si porta menys de 3 anys se li aplica un augment del 3%.

   L'antiguitat és una dada que se li demana a l'usuari per teclat.

5. Escriu un programa que demani a l'usuari un número entre l'1 i el 7, i la sortida per pantalla serà indicar el dia de la setmana a que correspon aquest número. **Heu de fer servir l'instrucció switch**.

6. Teniu un germà petit que està aprenent les taules de multiplicar i el voleu ajudar, prepareu un programa que li mostri multiplicacions i li demani per entrar el resultat i depenent si l'ha encertat o no mostreu un missatge felicitant-lo o deixant que ho torni a intentar com a màxim 5 cops i sinó mostrant-li el resultat.

7. Escriu un programa que demani un número de l'1 al 10 i escrigui per pantalla la taula de multiplicar d'aquest número.

8. Escriu un programa que demani la nota que ha tret el vostre germà a l'examen de les taules de multiplicar i mostri *suspès, suficient, bé, notable o excel · lent.*

9. Escriu un programa que demani a l'usuari un número entre l'1 i el 12, i la sortida per pantalla serà indicar el mes en text a que correspon aquest número. **No podeu fer servir un if/switch per donar resposta; utilitzeu un array estàtic d'strings.**

10. Escriu un programa que demani 10 números y els guardi en un array estàtic. Un cop guardats, indiqueu per pantalla quin és el més gran i quin és el més petit en aquest array.

11. Escriu un programa que demani entrar per consola 15 lletres, les guardi en un array estàtic i compti quantes vegades es troba repetida cada lletra.

12. Escriu un programa que demani un nombre n i creï un array dinàmic de n posicions, amb elements ordenats d'1 a n. Si la seva mida és parella, invertirà els elements de cada meitat de l'array. Per exemple: v=[1][2][3][4][5][6] v(invertit)=[3][2][1][6][5][4]

13. Dedueix quin és el resultat dels següents codis i raona per què:

    (a) Pas de paràmetres per valor/referència:

```
1  void increase1(int x){ x++; }
2  void increase2(int &x){ x++; }
3
4  int main() {
5      int x = 0;
6
7      increase1(x);
8      cout << x << "␣";
9      increase2(x);
10     cout << x << endl;
11
12     return 0;
13 }
```

(b) Retorn de valor/referència:

```
int increase3(int &x){ x++; return x; }
int& increase4(int &x){ x++; return x; }

int main() {
    int x = 0;
    int a = increase3(x);
    a++;
    cout << x << "␣" << a << endl;

    x = 0;
    int &b = increase4(x);
    b++;
    cout << x << "␣" << b << endl;

    x = 0;
    int c = increase4(x);
    c++;
    cout << x << "␣" << c << endl;

    return 0;
}
```

(c) Per què dona Warning la següent funció?

```
int& increase5(int x){ x++; return x; }
```

14. Dedueix quin és el resultat del següent codi i raona per què:

```
int increase(int *x) { (*x)++; return *x; }

int main() {
    int x = 1, y = 2;
    int *pX = &x, *pY = &y;
    // &x not a reference; & in this context is an operator to obtain x's address.

    cout << x << "␣" << pX << "␣" << *pX << endl;
    // *pX not a pointer; * in this context is an operator to obtain the content of
        pX's memory address.
    cout << y << "␣" << pY << "␣" << *pY << endl;

    x++; (*pY)++;
    cout << *pX << "␣" << y << endl;

    pX = pY; (*pX)++;
    cout << x << "␣" << y << endl;

    increase(&x); increase(pY);
    cout << x << "␣" << y << endl;

    return 0;
}
```

15. Quin dels dos checks (==) donarà true? Per què?

```
// Note: int **p means pointer of (a pointer of ints).
void f1(int x, int **p){ *p = &x; }
void f2(int &x, int **p){ *p = &x; }

int main(){
    int x = 1, *p1, *p2;

    f1(x, &p1); f2(x, &p2);
    cout << (&x == p1) << "␣" << (&x == p2) << endl;

    return 0;
}
```

2 Problemes de classes i objectes en C++

16. Crea la classe `Pacient` que contingui el `nom`, `edat`, `alçada` i `pes` del pacient. Crea un mètode *calcBMI* a la classe `Pacient` per calcular el `BMI` (Body Mass Index) amb l'alçada i el pes; i un mètode *evalBMI* que escrigui per pantalla si el BMI és molt baix, baix, normal, alt o molt alt.

    El càlcul és relativament simple: consisteix a dividir el pes, expressat en quilos, entre l'alçada, en metres, elevada al quadrat (kg/m2). El BMI resultant indica si el pes és correcte en funció d'aquesta alçada, o si és massa baix o elevat.

    - Generalment es considera satisfactori un BMI que se situa entre 18,5 i 24,9.
    - Si el BMI és inferior a 18,5, probablement sigui massa prim.
    - Si és igual o superior a 25, tens uns quilos de més.
    - Si el BMI està per sobre de 30, podem parlar d'obesitat.

    Atenció: l'BMI és vàlid només per a majors de 18 anys; no és aplicable per a adolescents si no s'introdueix un factor de correcció.

17. Definiu una clase `Time` que permeti guardar l'hora, minut i segon. L'hora ha d'estar en el rang de valors de 0 a 23, minut té un rang de valors de 0 a 59 i segon té un rang de 0 a 59. A la implementació de la classe, fareu els següents mètodes:

    - Un getter per a cada atribut
    - Una funció setter per a cada atribut
    - Un constructor per defecte per a la classe
    - Un constructor amb paràmetre per a la classe. Aquest constructor recollirà les dades pels tres atributs.
    - Una funció `nextSecond` que retornar un objecte Time amb un segon més que l'actual.
    - Una funció `print` que mostri per pantalla l'hora, minut i segon en el següent format: hh:mm:ss

    A més a més, implementeu un main.cpp que crei dues instàncies de Time. La primera d'elles ha d'estar creada estàticament amb el constructor amb paràmetres, la segona d'elles l'heu de crear dinàmicament. Afegiu dues hores a cada instància i imprimiu per pantalla el seu contingut.

18. Definiu la classe `Usuari`. La classe`Usuari` té dues dades membre: Nom d'Identificació de l'Usuari (conegut com ID) i el Nombre d'Identificació Personal (conegut com a PIN). Per a efectes d'aquest exercici, es considerarà vàlid un ID que tingui una longitud mínima de 6 caràcters; i en el cas del PIN el valor vàlid serà qualsevol nombre major que zero. Implementeu els següents mètodes a la classe:

    - Una función setter per introduir les dues dades membre.
    - Una funció setter per a cada dada membre.
    - Un constructor per defecte per a la classe.
    - Un constructor amb paràmetres per a la classe. Aquest constructor recollirà les dades per tots els atributs.

- Una funció `print` que mostri per pantalla les dades membre de l'objecte.

A més a més, implementeu un main.cpp que faci les següents accions:

- Crear un array de punters de Usuari de 4 posicions (és a dir, array de *Usuari). Afegir els següents objectes:
  - Definir i inicialitzar un primer objecte estàticament amb el constructor per defecte
  - Definir i inicialitzar un segon objecte estàticament amb el constructor amb paràmetres amb valors arbitraris.
  - Definir i inicialitzar un tercer objecte dinàmicament amb el constructor per defecte
  - Definir i inicialitzar un quart objecte dinàmicament amb el constructor amb paràmetres amb valors arbitraris.
  - Demanar les dades pel teclat per definir valors als objectes construïts amb el constructor per defecte.
- Imprimir la informació de cada objecte de l'array.

19. Crea un projecte i definiu una clase `Complex` que permeti emmagatzemar la part real i la part imaginària d'un nombre. Els valors per defecte estaran a 0.0 i els atributs de la classe són privats. Es necessari implementar els següents mètodes a la classe:

    - Un getter per a cada atribut.
    - Una funció setter per a cada atribut.
    - Un constructor per defecte per a la classe.
    - Un constructor amb paràmetre per a la classe. Aquest constructor demanarà les dades pels dos atributs.
    - Una funció `void add(Complex & another)` que permeti sumar la informació d'un complex al complex actual (this).
    - Una funció `Complex addReturnNew(double re, double im) const` que permeti retornar una nova instància del complex amb la suma. L'actual (this) no ha de canviar.
    - Una funció `print` que mostri per pantalla les dades del complex (real, imag).

    A més a més, implementeu el main.cpp que usi aquesta nova clase Complex i que faci:

    - Crear un array estàtic de 10 instàncies de tipus Complex, omplir-lo amb complexos arbitraris, i fer print de cada nombre.
    - Definir i inicialitzar un objecte Complex dinàmicament amb el constructor per defecte, que esdevingui la suma dels 10 complexos continguts a l'array anterior. Fer print d'aquest nou nombre.
    - Obtenir un nou Complex a memòria estàtica que contingui la resta de la suma anterior menys el nombre $(35, 35)$. Feu print d'aquest nou nombre. També feu print de nou de la suma anterior, per comprovar que no s'ha modificat el seu valor. Noteu que restar (x, y) equival a sumar $(-x, -y)$.

20. Crea un projecte que contingui tres classes. La classe `Pacient` definida a l'exercici anterior, una nova classe anomenada `LlistatPacients` que permeti emmagatzemar Pacients en un `vector<Pacient*>` i tingui tres mètodes: un mètode per anomenat `afegirPacient(Pacient *pac)`, un mètode `eliminarPacient(string nom)`, i un mètode `print()` (que llista els noms dels pacients). En el `main` es farà un petit menú amb tres opcions:

```
1. Afegir pacient.
2. Eliminar pacient.
3. Llistar pacients.
```

Les dades s'introduiran per teclat.

NOTA: Heu de fer el destructor de LlistatPacients per alliberar els punters de Pacient que hi han al vector.

21. Modifica el problema anterior i incorpora una quarta opció de menú que permeti abocar tota la informació dels pacients en un fitxer de text. El nom del fitxer a abocar es demanarà a l'usuari per teclat. Cada fila del fitxer de sortida tindrà la informació d'un pacient, separant cada camp (atribut) per un tabulador.

22. Modifica el problema anterior, permetent que tingui una cinquena opció al menú que permeti afegir pacients des d'un fitxer de text. El nom del fitxer es llegeix des de teclat i cada fila del fitxer contindrà la informació d'un pacient. S'han d'introduir tots els pacients del fitxer a `LlistatPacients.cpp`